1. Introduction

Pour ce projet, vous devez implémenter un jeu de course de voitures en 2D, inspiré de Mario Kart et de Trackmania. Vous devez suivre les instructions données dans ce document.

Les objectifs de ce projet sont :
- Pratiquer la programmation orientée objet vue en cours
- Acquérir de l'expérience en programmation Python
- Apprendre à collaborer sur des projets de programmation

Calendrier :
- 9/10 : date limite pour envoyer la composition de vos groupes
- 12/10 : annonce des groupes
- 4/12 : date limite de rendu du projet (dépôt GitHub)
- 11/12 : examen oral

Plagiat : Nous aurons accès à tous les projets. Il sera donc facile de détecter du plagiat ou une collaboration entre différents groupes. Cela serait bien entendu lourdement sanctionné.

LLM : Vous avez le droit d'utiliser des LLM pour vous aider à écrire votre code. Mais attention :
- Vous devez relire tout ce qui a été généré et reformuler les commentaires si nécessaire. Par exemple, les LLM écrivent souvent dans le code des commentaires destinés à l'utilisateur du LLM. Vous devez supprimer ou réécrire ces commentaires.
- Vous devez vous assurer que vous comprenez **tout** ce qui a été généré. Lors de l'examen oral, nous vous demanderons d'expliquer des parties de votre code. Si vous ne parvenez pas à l'expliquer clairement, nous en déduirons qu'il a été généré et que vous n'avez pas fait l'effort de bien le comprendre. Là encore, vous seriez pénalisés.
- Votre fichier README doit contenir une déclaration expliquant comment vous avez utilisé des outils d'IA dans le projet. Vous devez indiquer si vous avez utilisé une IA :
  - Pour traduire ou expliquer le sujet
  - Pour vous aider dans la phase de conception
  - Pour écrire du code
  - Pour le débogage
  - Pour écrire des commentaires et des docstrings
  - Pour dessiner le diagramme de classes UML

2. Description du jeu

Le jeu que vous devez implémenter est un jeu de course de voitures simple à deux joueurs, inspiré de Trackmania et de Mario Kart. Les joueurs contrôlent l'angle de braquage et l'accélération des voitures, et le but est d'atteindre le premier la fin de la course.

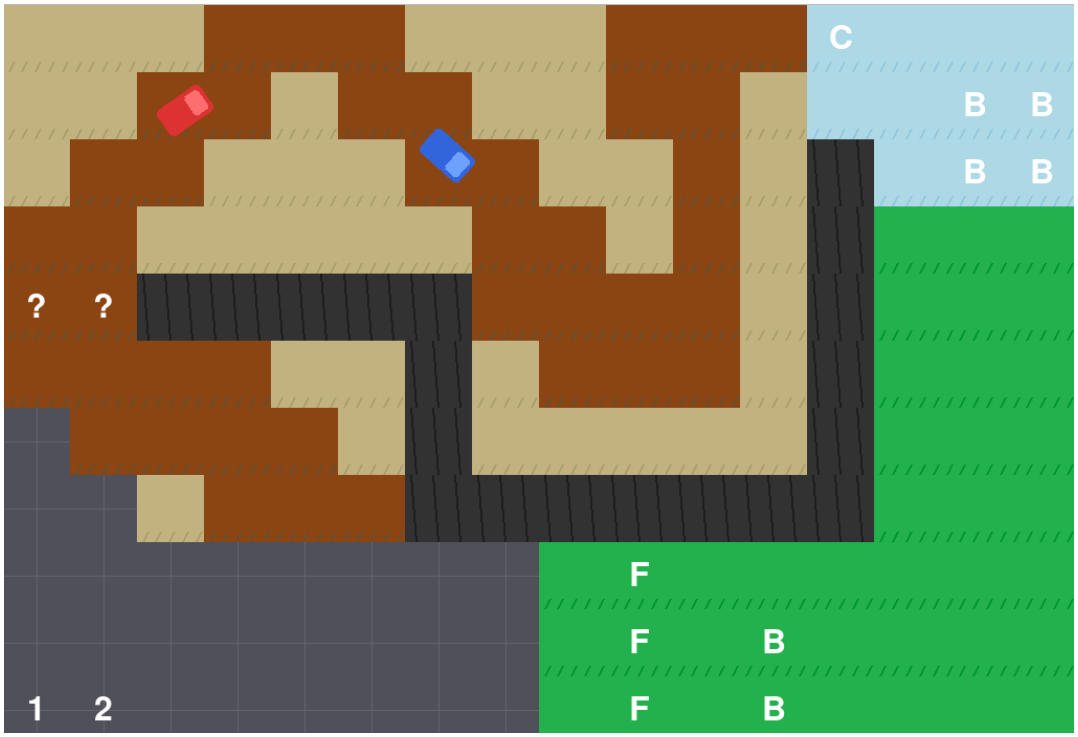


Figure 1 – Capture d'écran du jeu. Les couleurs des cases indiquent le type de terrain, tandis que les lettres blanches indiquent le type de bloc. Votre jeu n'a pas besoin de ressembler à celui-ci. Vous êtes libres de choisir des visuels très différents (de préférence plus beaux que celui-ci).

2.1. **Commandes.** Les deux voitures doivent être contrôlées à l'aide d'un seul clavier :

- Avancer / accélérer : Z ou W (joueur 1) et flèche du haut (joueur 2)
- Reculer / freiner : S (joueur 1) et flèche du bas (joueur 2)
- Tourner à gauche : Q ou A (joueur 1) et flèche de gauche (joueur 2)
- Tourner à droite : D (joueur 1) et flèche de droite (joueur 2)
- Utiliser un objet : barre d'espace (joueur 1) et Entrée (joueur 2)

De plus, l'interface de votre jeu doit permettre aux joueurs de naviguer dans des menus (par exemple pour choisir le circuit).

2.2. **Dynamique des voitures.** Comme dans Trackmania et Mario Kart, nous voulons que les voitures puissent déraper (drift). Dans le jeu que vous devez implémenter, le dérapage se produit lorsque l'orientation de la voiture n'est pas alignée avec sa vitesse (voir figure 2). Si vous connaissez ces deux jeux, ce n'est pas comme dans Mario Kart où une commande spécifique permet de déclencher le dérapage, mais plutôt proche du comportement de Trackmania. La propension à déraper dépend du type de terrain sur lequel se trouve la voiture.

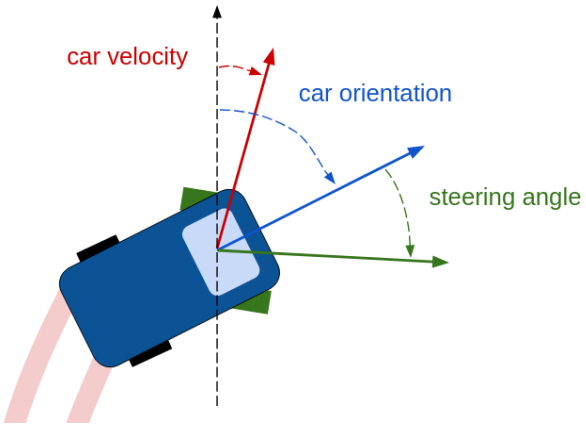


FIGURE 2 – Dynamique de la voiture.

Par exemple, il doit être plus facile de déraper sur l'herbe que sur la route, et il doit être presque impossible de ne pas déraper sur la glace. Il existe d'autres terrains, comme la terre et le sable, représentés sur la figure 1.

Les commandes de l'utilisateur agissent sur la dynamique de la voiture selon les principes généraux suivants :

- Avancer / accélérer : fixe une valeur positive pour l'accélération longitudinale de la voiture.
- Reculer / freiner : fixe une valeur négative pour l'accélération longitudinale de la voiture.
- Tourner à gauche : augmente l'angle de braquage de la voiture (convention trigonométrique).
- Tourner à droite : diminue l'angle de braquage de la voiture.

Ensuite, la vitesse, l'orientation et l'état de dérapage de la voiture (si la voiture est en train de déraper ou si elle adhère correctement à la piste) sont mis à jour en fonction de l'accélération longitudinale, de l'angle de braquage et du type de terrain.

La dynamique doit également inclure des frottements :

- Frottements avec l'air.
- Frottements avec le terrain lorsque la voiture dérape.

Il existe de nombreuses façons d'implémenter cela, vous n'avez pas besoin de suivre exactement ces étapes. Tant que vous avez une voiture capable d'avancer, de tourner et de déraper, cela devrait convenir. Faites attention à ce que votre voiture se comporte aussi correctement en marche arrière.

2.3. **Circuits et terrains.** Le circuit est une grille 2D de cases qui peuvent correspondre à différents terrains et qui peuvent en plus contenir certains blocs.

Premièrement, les terrains peuvent avoir plusieurs effets sur la dynamique des voitures :

- Différents seuils de déclenchement du dérapage : il doit être plus facile de déraper sur la glace que sur l'herbe, plus facile de déraper sur l'herbe que sur la route, etc.
- Différents niveaux de frottement lorsque la voiture dérape. Par exemple, la glace est très glissante, il n'y a donc presque aucun frottement lorsque la voiture dérape.

— Différents niveaux de traction, la traction étant ici un coefficient qui détermine la facilité avec laquelle on peut accélérer ou freiner sur le terrain. Par exemple, la route doit avoir une très bonne traction, tandis que la terre doit avoir une mauvaise traction.
— Enfin, certains terrains peuvent être infranchissables (comme des murs, des arbres, etc.). Cela ne fait pas partie des fonctionnalités de base obligatoires.

Deuxièmement, les cases peuvent contenir des blocs ayant différents effets :

— Points de passage (notés « C » sur la figure 1) : la voiture doit passer sur tous les points de passage avant d'atteindre la ligne d'arrivée. Chaque circuit possède un nombre défini de points de passage.
— Lignes d'arrivée (notées « L » sur la figure 1) : passer sur la ligne d'arrivée après avoir franchi tous les points de passage augmente le compteur de tours du joueur. Chaque circuit possède un nombre défini de tours que les joueurs doivent effectuer pour gagner la course.
— Boosts (notés « B » sur la figure 1) : passer sur un bloc de boost donne un bonus d'accélération.
— Blocs d'objet (notés « ? » sur la figure 1) : passer sur un bloc d'objet donne un objet au joueur s'il n'en possède pas déjà un. Les objets et les blocs d'objet ne font pas partie des fonctionnalités de base obligatoires.
— Blocs de départ (notés « 1 » et « 2 » sur la figure 1) : ces blocs indiquent les positions de départ des deux joueurs.

2.4. **Interface utilisateur.** L'interface du jeu doit afficher le circuit et les voitures, comme représenté sur la figure 1. Les objets actuellement possédés par les joueurs, ainsi que les effets des objets, doivent également apparaître à l'écran.

L'interface utilisateur doit aussi comporter un menu de navigation permettant aux joueurs (entre autres fonctionnalités possibles) de choisir le circuit.

2.5. **Fonctionnalités de base (pour tous).** Cette section décrit les fonctionnalités qui doivent être implémentées dans votre programme. Des fonctionnalités plus avancées sont décrites dans la section suivante et ne concernent que les étudiants SI, ROB et MEDH.

— Dynamique des voitures :
  — Pouvoir accélérer / freiner.
  — Pouvoir tourner.
  — Déclenchement automatique du dérapage lorsque les conditions sont réunies.
  — Dynamique réaliste de la voiture lorsqu'elle ne dérape pas.
  — Dynamique réaliste de la voiture lorsqu'elle dérape.
— Terrains :
  — Route : traction élevée, difficile de déraper, frottements très importants en dérapage.
  — Herbe : traction moyenne, facile de déraper, frottements importants en dérapage.
  — Terre : traction moyenne, très facile de déraper, frottements intermédiaires en dérapage.
  — Glace : faible traction, difficile de ne pas déraper, presque aucun frottement en dérapage.
  — Vide : lorsque les voitures passent sur un bloc de vide, elles sont téléportées à leur dernier point de passage, avec une vitesse nulle. L'extérieur de la carte est considéré comme du vide.

- Blocs :
  - Points de passage.
  - Lignes d'arrivée.
  - Boosts.
  - Blocs de départ.
- Interface graphique :
  - Afficher le circuit.
  - Afficher les voitures.
  - Afficher les objets et les effets des objets.
  - Menu de navigation.

2.6. **Fonctionnalités supplémentaires (SI, ROB, MEDH uniquement).**

- Objets :
  - Bloc d'objet donnant un objet aléatoire au joueur, avec des probabilités différentes pour chaque catégorie d'objets :
    - Commun : probabilité élevée
    - Rare : probabilité moyenne
    - Épique : probabilité faible
    - Légendaire : probabilité très faible
  - Champignon : donne un boost à la voiture. Commun.
  - Banane : dépose une banane sur la piste. La prochaine voiture qui passe sur la banane sera ralentie. Commun.
  - Bombe : dépose une bombe sur la piste. La bombe explose après un court délai et arrête toutes les voitures à proximité. Rare.
  - Étoile : rend invulnérable pendant 10 secondes. Épique.
  - Carapace bleue : fait tomber une bombe directement sur l'adversaire. Légendaire.
- Personnalisation des voitures : possibilité de choisir une voiture dans le menu. Les voitures auront des valeurs différentes pour les propriétés suivantes :
  - Accélération : influe sur l'accélération longitudinale lorsque l'on appuie sur la touche pour avancer.
  - Maniabilité : influe sur la vitesse de braquage (la capacité à tourner serré dans les virages).
  - Design : purement esthétique, chaque voiture doit avoir un design différent.
  - Aérodynamisme : influe sur la sensibilité aux frottements de l'air.
- Blocs à effet spécial :
  - Direction bloquée : il n'est plus possible de tourner jusqu'à atteindre un point de passage ou un bloc de réinitialisation.
  - Freins bloqués : il n'est plus possible de freiner jusqu'à atteindre un point de passage ou un bloc de réinitialisation.
  - Moteur coupé : il n'est plus possible d'accélérer jusqu'à atteindre un point de passage ou un bloc de réinitialisation.
  - Réinitialisation : annule tous les effets (comme direction bloquée, freins bloqués, moteur coupé).
- Possibilité de revenir au dernier point de passage : ajouter une touche permettant au joueur de revenir à son dernier point de passage. Utile pour ne pas rester bloqué à cause des blocs à effet spécial.

— Objets supplémentaires : vous pouvez vous inspirer des objets de Mario Kart.

3. Point de départ

Vous ne partirez pas de zéro. Un point de départ pour le projet vous est fourni. Votre première étape doit être d'examiner attentivement le code fourni pour comprendre comment il est structuré.

Le code est structuré selon le patron de conception MVC. Ce patron de conception vise à séparer clairement les parties du code qui gèrent le modèle, la vue et le contrôleur :

— (M) Modèle : partie du code qui implémente vos structures de données et la logique de votre programme. C'est là que vous devriez implémenter la plupart de vos classes : la dynamique des voitures, le circuit, les différents terrains, les blocs, etc.
— (V) Vue : partie du code qui affiche l'état de votre modèle à l'écran, ici à l'aide de la bibliothèque `pygame`.
— (C) Contrôleur : partie du code qui traite les entrées de l'utilisateur et met à jour le modèle et la vue en conséquence.

Le code fourni utilise la bibliothèque `pygame`. Vous devrez d'abord installer `pygame` avant de pouvoir lancer le programme. Vous pourrez ensuite lire les parties du code qui utilisent `pygame` pour comprendre de manière générale comment utiliser la bibliothèque. Vous pouvez aussi consulter la documentation en ligne si nécessaire.

4. Évaluation

Vous recevrez trois évaluations différentes (/20) selon les critères suivants :

(1) La qualité du programme lui-même.
(2) La qualité du code et de la documentation (commentaires, docstrings, diagramme).
(3) Votre implication dans le projet et votre compréhension du code.

La note finale correspond au minimum de ces trois notes.

Les situations suivantes mèneront donc inévitablement à de mauvaises notes :

— Votre groupe a rendu un très bon programme, le code est clairement écrit et bien documenté, mais vous n'avez pas été très actif dans le projet.
— Votre groupe a rendu un très bon programme, vous avez été très actif dans l'écriture du code, mais le code est mal écrit, mal documenté, et n'utilise pas correctement les principes de la programmation orientée objet.

4.1. **Votre programme.** Cette note évalue la qualité du jeu, de l'interface utilisateur et des fonctionnalités. Le barème est donné dans deux tableaux distincts. Le premier tableau donne le barème des fonctionnalités de base, qui doivent être implémentées par tous. Le second tableau donne les fonctionnalités avancées supplémentaires et leur barème pour les groupes SI, ROB et MEDH.

Les groupes CIMES, IPS et SYSCOM doivent viser 20 points pour cette note. Les groupes SI, ROB et MEDH doivent viser 30 points, qui seront ramenés à une note sur 20. Il n'y aura pas

de bonus pour les groupes CIMES, IPS et SYSCOM qui implémenteraient les fonctionnalités supplémentaires.

Fonctionnalités de base (pour tous)	**Points**
Qualité de l'interface graphique	4
Dynamique des voitures (accélérer, freiner, tourner, mécanismes de dérapage)	4
Blocs (bloc de départ, points de passage, boosts, ligne d'arrivée)	4
Terrains (route, herbe, terre, glace, vide)	4
Impression générale (le jeu est-il agréable et facile à prendre en main)	4

TABLE 1 – Barème des fonctionnalités de base. Le barème totalise 20 points. **Tous les groupes doivent implémenter ces fonctionnalités.**

Fonctionnalités supplémentaires (SI, ROB, MEDH)	**Points**
Objets (bloc d'objet, champignon, banane, bombe, étoile, carapace bleue)	4
Personnalisation des voitures (menu, accélération, maniabilité, design, etc.)	4
Blocs à effet spécial (direction bloquée, freins bloqués, moteur coupé, etc.)	3
Retour au dernier point de passage	1
Terrains infranchissables (mur, rocher, arbre) avec implémentation des collisions	2
Tout objet supplémentaire suffisamment différent des autres	1
Tout bloc à effet spécial supplémentaire suffisamment différent des autres	1
Toute propriété de terrain supplémentaire suffisamment différente des autres	1
Toute propriété de voiture supplémentaire suffisamment différente des autres	1

TABLE 2 – Barème des fonctionnalités supplémentaires. **Ce tableau ne concerne que les groupes SI, ROB, MEDH**.

4.2. **Qualité du code et documentation.** Le code et la documentation seront évalués différemment pour CIMES/IPS/SYSCOM et pour SI/ROB/MEDH.

Le premier tableau donne le barème pour les groupes CIMES/IPS/SYSCOM :

Critère d'évaluation (CIMES, IPS, SYSCOM)	**Points**
Clarté et propreté du code (respect des conventions Python)	5
Commentaires et docstrings dans les classes et les méthodes	5
Dépôt (requirements.txt, instructions dans le README pour installer et lancer)	5
Utilisation pertinente des classes, attributs, méthodes pour structurer votre code	5

TABLE 3 – Barème pour les groupes **CIMES, IPS, SYSCOM**.

Pour les groupes SI/ROB/MEDH, votre dépôt GitHub doit contenir un diagramme de classes présentant toutes les classes principales de votre programme et leurs relations (héritage, composition, etc.).

Dans ce projet, vous n'aurez pas besoin d'encapsuler vos classes. Vous pouvez garder toutes vos méthodes et tous vos attributs publics si vous le souhaitez. Cela n'aura aucun effet sur l'évaluation.

Le tableau suivant donne le barème pour les groupes SI, ROB, MEDH :

Critère d'évaluation (SI, ROB, MEDH)	**Points**
Clarté et propreté du code (respect des conventions Python)	3
Commentaires et docstrings dans les classes et les méthodes	3
Dépôt (requirements.txt, instructions dans le README pour installer et lancer)	3
Utilisation pertinente des classes, attributs, méthodes pour structurer votre code	3
Utilisation pertinente de l'héritage (au moins deux classes parentes différentes dans votre programme)	2
Utilisation pertinente de l'abstraction (au moins une classe abstraite dans votre programme)	2
Qualité et complétude du diagramme de classes UML	2
Cohérence entre le diagramme de classes UML et le code	2

TABLE 4 – Barème pour les groupes **SI, ROB, MEDH**.

4.3. **Implication et compréhension.** Cette note **est individuelle** et évalue votre implication dans le groupe, votre bonne utilisation de git, et votre compréhension personnelle du code à la fin du projet.

Chaque groupe devra créer un dépôt GitHub public **au début** du projet, et contribuer à ce dépôt tout au long du projet. Pour évaluer votre implication, nous vérifierons que vous n'avez pas commencé à contribuer au projet juste avant la date limite, que vous avez fait des commits fréquemment sur le dépôt, que les noms de vos commits sont informatifs, et que vos contributions sont significatives. C'est en effet une bonne habitude d'enregistrer régulièrement votre progression avec des commits : n'attendez pas la fin du projet pour créer le dépôt et y déposer tout votre code. Tous les membres du groupe doivent participer sur GitHub. Si vous n'avez pas participé sur le dépôt, nous considérerons que vous n'avez pas participé au projet (même si vous pouvez prouver que vous avez participé et envoyé votre code par d'autres moyens).

Attention : il ne s'agit pas d'une course pour participer plus que les autres au projet. Si vous avez implémenté 1/3 du programme (dans un groupe de 3), c'est suffisant. Il n'y aura aucun bonus pour quelqu'un qui réalise 50 % ou même 90 % du projet seul. À l'inverse, si vous avez l'impression que votre groupe vous empêche de faire votre part du travail (par exemple : implémenter les fonctionnalités que vous deviez implémenter), vous pouvez me contacter et j'arbitrerai la situation.

Enfin, nous voulons évaluer votre compréhension personnelle du code que vous avez écrit. Un examen oral évaluera en particulier si vous comprenez toutes les parties du code, et si vous êtes capables de discuter de tous vos choix de conception.

Attention : vous pouvez être interrogés sur n'importe quelle partie du code, y compris des parties que vous n'avez pas écrites (écrites par vos camarades, par une IA, ou fournies comme point de départ). Cela fait partie de votre travail de lire et de comprendre les commits de vos camarades, et si nécessaire de leur demander des explications.

Critères d'évaluation	**Points**
Bonne utilisation de git, commits fréquents, contribution significative au projet	10
Examen oral	10

TABLE 5 – Barème de la note individuelle.